Smart Used Car Price Estimation System

Code and Output | Machine Learning Laboratory (CS3201L) | TY Div A | 2026-27

Step 1: Libraries and settings

Code

```
import os
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, DBSCAN
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score, KFold
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor, NearestNeighbors
from sklearn.ensemble import AdaBoostRegressor
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

warnings.filterwarnings("ignore")

CSV_PATH = "cars.csv"
NAME_COL = "name"
YEAR_COL = "year"
PRICE_COL = "selling_price"
KM_COL = "km_driven"
FUEL_COL = "fuel"
SELLER_COL = "seller_type"
TRANS_COL = "transmission"
OWNER_COL = "owner"
CURRENT_YEAR = 2026
RANDOM_STATE = 42
SEGMENT_NAMES = ["Budget", "Mid-range", "Premium"]
OWNER_MAP = {"test drive car": 0, "first owner": 1, "second owner": 2,
             "third owner": 3, "fourth & above owner": 4}
```

Step 2: Data loading and cleaning

Code

```
def make_synthetic(n=1500):
    rng = np.random.default_rng(RANDOM_STATE)
    brands = {"Maruti": 4.5, "Hyundai": 5.5, "Honda": 7.0, "Tata": 6.0, "Ford": 6.5,
              "Mahindra": 9.0, "Toyota": 10.0, "BMW": 35.0, "Audi": 38.0, "Mercedes": 40.0}
    weights = np.array([0.24, 0.2, 0.1, 0.1, 0.07, 0.09, 0.09, 0.04, 0.03, 0.04])
    owners = ["First Owner", "Second Owner", "Third Owner", "Fourth & Above Owner"]
    owner_factor = {"First Owner": 1.0, "Second Owner": 0.92, "Third Owner": 0.85,
                    "Fourth & Above Owner": 0.8}
    fuel_factor = {"Petrol": 1.0, "Diesel": 1.12, "CNG": 0.9, "LPG": 0.85}
    rows = []
    names = list(brands)
    for _ in range(n):
        brand = rng.choice(names, p=weights / weights.sum())
        year = int(rng.integers(2008, 2026))
        age = CURRENT_YEAR - year
        km = int(max(2000, age * rng.uniform(7000, 15000)))
        fuel = rng.choice(list(fuel_factor), p=[0.5, 0.42, 0.06, 0.02])
        trans = rng.choice(["Manual", "Automatic"], p=[0.8, 0.2])
        seller = rng.choice(["Individual", "Dealer", "Trustmark Dealer"], p=[0.7, 0.25, 0.05])
        owner = rng.choice(owners, p=[0.65, 0.25, 0.08, 0.02])
        price = (brands[brand] * np.exp(-0.11 * age) * np.exp(-0.000004 * km)
                 * fuel_factor[fuel] * (1.15 if trans == "Automatic" else 1.0)
                 * owner_factor[owner] * np.exp(rng.normal(0, 0.12)))
        model = rng.choice(["Alpha", "Beta", "Gamma"])
        rows.append([f"{brand} {model}", year, int(price * 100000), km, fuel, seller, trans, owner])
    df = pd.DataFrame(rows, columns=[NAME_COL, YEAR_COL, PRICE_COL, KM_COL, FUEL_COL,
                                     SELLER_COL, TRANS_COL, OWNER_COL])
    df.loc[rng.choice(n, 20, replace=False), KM_COL] = np.nan
    return df


def load_data():
    if os.path.exists(CSV_PATH):
        df = pd.read_csv(CSV_PATH)
        print("Loaded dataset:", CSV_PATH)
    else:
```

```
        df = make_synthetic()
        print("cars.csv not found, using generated sample data")
    df = df[[NAME_COL, YEAR_COL, PRICE_COL, KM_COL, FUEL_COL, SELLER_COL, TRANS_COL, OWNER_COL]].copy()
    df = df.drop_duplicates().reset_index(drop=True)
    for c in [YEAR_COL, PRICE_COL, KM_COL]:
        df[c] = pd.to_numeric(df[c], errors="coerce")
    df = df.dropna(subset=[NAME_COL, YEAR_COL, PRICE_COL]).reset_index(drop=True)
    df[KM_COL] = df[KM_COL].fillna(df[KM_COL].median())
    for c in [FUEL_COL, SELLER_COL, TRANS_COL]:
        df[c] = df[c].fillna("Unknown").astype(str)
    owner_text = df[OWNER_COL].astype(str).str.strip().str.lower()
    df["owner_num"] = pd.to_numeric(owner_text.map(OWNER_MAP), errors="coerce")
    df["owner_num"] = df["owner_num"].fillna(pd.to_numeric(owner_text, errors="coerce"))
    df["owner_num"] = df["owner_num"].fillna(1)
    low, high = df[PRICE_COL].quantile([0.01, 0.99])
    df = df[(df[PRICE_COL] >= low) & (df[PRICE_COL] <= high)]
    df = df[df[KM_COL] <= df[KM_COL].quantile(0.995)].reset_index(drop=True)
    df["brand"] = df[NAME_COL].astype(str).str.split().str[0]
    df["car_age"] = (CURRENT_YEAR - df[YEAR_COL]).clip(lower=1)
    df["km_per_year"] = df[KM_COL] / df["car_age"]
    df["km_per_year"] = df["km_per_year"].clip(upper=df["km_per_year"].quantile(0.99))
    df["price_lakh"] = df[PRICE_COL] / 100000
    return df


def build_features(df, brand_map, fallback, columns=None):
    X = pd.DataFrame({
        "car_age": df["car_age"].values,
        "km_driven": df[KM_COL].values,
        "km_per_year": df["km_per_year"].values,
        "owner_num": df["owner_num"].values,
        "brand_value": df["brand"].map(brand_map).fillna(fallback).values,
    })
    dummies = pd.get_dummies(df[[FUEL_COL, SELLER_COL, TRANS_COL]].astype(str)).astype(int)
    dummies.index = X.index
    X = pd.concat([X, dummies], axis=1)
    if columns is not None:
        X = X.reindex(columns=columns, fill_value=0)
    return X


def scores(y_true, y_pred):
    return {"R2": r2_score(y_true, y_pred),
            "MAE": mean_absolute_error(y_true, y_pred),
            "RMSE": np.sqrt(mean_squared_error(y_true, y_pred))}
```

Step 3: Train the models

Code

```
def train_models(df):
    train_df, test_df = train_test_split(df, test_size=0.2, random_state=RANDOM_STATE)
    train_df = train_df.reset_index(drop=True)
    test_df = test_df.reset_index(drop=True)
    brand_map = train_df.groupby("brand")["price_lakh"].median()
    fallback = train_df["price_lakh"].median()
    X_train = build_features(train_df, brand_map, fallback)
    columns = list(X_train.columns)
    X_test = build_features(test_df, brand_map, fallback, columns)
    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)
    y_train = np.log(train_df["price_lakh"].values)
    y_test = test_df["price_lakh"].values
    results = []
    models = {}

    mlr = LinearRegression().fit(X_train_s, y_train)
    models["Multiple Linear Regression"] = (mlr, lambda X: X)

    poly = PolynomialFeatures(degree=2, include_bias=False)
    poly.fit(X_train_s[:, :5])

    def poly_tf(X):
        return np.hstack([poly.transform(X[:, :5]), X[:, 5:]])

    pr = LinearRegression().fit(poly_tf(X_train_s), y_train)
    models["Polynomial Regression"] = (pr, poly_tf)

    tree = DecisionTreeRegressor(max_depth=6, random_state=RANDOM_STATE).fit(X_train_s, y_train)
    models["Decision Tree"] = (tree, lambda X: X)
```

```
    kf = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    ks = list(range(1, 26, 2))
    k_rmse = []
    for k in ks:
        s = cross_val_score(KNeighborsRegressor(n_neighbors=k), X_train_s, y_train,
                            cv=kf, scoring="neg_mean_squared_error")
        k_rmse.append(float(np.sqrt(-s.mean())))
    best_k = ks[int(np.argmin(k_rmse))]
    knn = KNeighborsRegressor(n_neighbors=best_k).fit(X_train_s, y_train)
    models["k-NN Regression"] = (knn, lambda X: X)

    grid = GridSearchCV(
        AdaBoostRegressor(estimator=DecisionTreeRegressor(max_depth=4),
                          random_state=RANDOM_STATE),
        {"n_estimators": [50, 100], "learning_rate": [0.1, 0.5]},
        cv=3, scoring="neg_mean_squared_error", n_jobs=-1)
    grid.fit(X_train_s, y_train)
    models["AdaBoost Regressor"] = (grid.best_estimator_, lambda X: X)

    preds = {}
    for name, (model, tf) in models.items():
        pred = np.exp(model.predict(tf(X_test_s)))
        preds[name] = pred
        row = {"Model": name}
        row.update(scores(y_test, pred))
        results.append(row)
    res = pd.DataFrame(results)
    print("\nBest K for k-NN (5-fold cross-validation):", best_k)
    print("Best AdaBoost settings:", grid.best_params_)
    print("\nModel comparison (price in lakh rupees):")
    print(res.round(3).to_string(index=False))
    best = res.loc[res["RMSE"].idxmin(), "Model"]
    print("\nBest model by RMSE:", best)
    return dict(train=train_df, test=test_df, models=models, scaler=scaler, columns=columns,
                brand_map=brand_map, fallback=fallback, res=res, best=best, preds=preds,
                ks=ks, k_rmse=k_rmse, best_k=best_k, y_test=y_test, X_train_s=X_train_s)
```

Step 4: Segments, unusual listings and new car prediction

Code

```
def segments_and_outliers(df):
    feats = np.column_stack([np.log(df["price_lakh"]), df["car_age"], df[KM_COL]])
    sc = StandardScaler()
    Z = sc.fit_transform(feats)
    km = KMeans(n_clusters=3, n_init=10, random_state=RANDOM_STATE).fit(Z)
    order = pd.Series(df["price_lakh"].values).groupby(km.labels_).mean().sort_values().index
    remap = {old: new for new, old in enumerate(order)}
    seg = np.array([remap[c] for c in km.labels_])
    db = DBSCAN(eps=0.45, min_samples=10).fit(Z)
    unusual = db.labels_ == -1
    print("\nCar segments:")
    for s in range(3):
        m = seg == s
        print(f"{SEGMENT_NAMES[s]}: {int(m.sum())} cars, average price {df.loc[m, 'price_lakh'].mean():.2f} lakh")
    print("\nUnusual listings found by DBSCAN:", int(unusual.sum()), "of", len(df))
    return seg, unusual, km, sc, remap


def predict_car(info, state, seg_pack, df):
    km_model, sc2, remap = seg_pack
    row = pd.DataFrame([{NAME_COL: info["name"], KM_COL: info["km"], FUEL_COL: info["fuel"],
                         SELLER_COL: info["seller"], TRANS_COL: info["transmission"]}])
    row["brand"] = row[NAME_COL].str.split().str[0]
    row["car_age"] = max(CURRENT_YEAR - info["year"], 1)
    row["km_per_year"] = row[KM_COL] / row["car_age"]
    row["owner_num"] = info["owner"]
    X = build_features(row, state["brand_map"], state["fallback"], state["columns"])
    Xs = state["scaler"].transform(X)
    model, tf = state["models"][state["best"]]
    price = float(np.exp(model.predict(tf(Xs))[0]))
    z = sc2.transform([[np.log(price), row["car_age"].iloc[0], info["km"]]])
    segment = SEGMENT_NAMES[remap[int(km_model.predict(z)[0])]]
    nn = NearestNeighbors(n_neighbors=3).fit(state["X_train_s"])
    _, idx = nn.kneighbors(Xs)
    similar = state["train"].iloc[idx[0]][[NAME_COL, YEAR_COL, KM_COL, "price_lakh"]]
    print(f"\nCar: {info['name']} {info['year']}, {info['km']} km, {info['fuel']}, {info['transmission']}")
    print(f"Predicted price: {price:.2f} lakh rupees ({state['best']})")
    print("Segment:", segment)
    print("Three most similar cars in the data:")
    print(similar.round(2).to_string(index=False))
```

Step 5: Graph functions

Code

```
def plot_actual_vs_pred(state):
    y, p = state["y_test"], state["preds"][state["best"]]
    plt.figure(figsize=(6, 6))
    plt.scatter(y, p, s=14, alpha=0.6)
    top = max(y.max(), p.max())
    plt.plot([0, top], [0, top], color="green", label="Perfect prediction")
    plt.xlabel("Actual price (lakh)")
    plt.ylabel("Predicted price (lakh)")
    plt.title("Actual vs Predicted Price: " + state["best"])
    plt.legend()
    plt.grid(True)
    plt.tight_layout()


def plot_age_curve(df):
    plt.figure(figsize=(8, 5))
    plt.scatter(df["car_age"], df["price_lakh"], s=8, alpha=0.4, label="Cars")
    coef = np.polyfit(df["car_age"], df["price_lakh"], 2)
    xs = np.linspace(df["car_age"].min(), df["car_age"].max(), 100)
    plt.plot(xs, np.polyval(coef, xs), color="red", linewidth=2, label="Polynomial curve (degree 2)")
    plt.xlabel("Car age (years)")
    plt.ylabel("Price (lakh)")
    plt.title("Price Drops as the Car Gets Older")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()


def plot_k_curve(state):
    plt.figure(figsize=(7, 4))
    plt.plot(state["ks"], state["k_rmse"], marker="o")
    plt.axvline(state["best_k"], color="red", linestyle="--", label=f"Best K = {state['best_k']}")
    plt.xlabel("K (number of neighbours)")
    plt.ylabel("Cross-validation error")
    plt.title("k-NN Regression: Effect of K")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()


def plot_models(res):
    fig, ax = plt.subplots(1, 2, figsize=(12, 5))
    ax[0].bar(res["Model"], res["R2"], color="#1F77B4")
    ax[0].set_title("R2 Score (higher is better)")
    ax[0].set_ylim(0, 1.05)
    ax[1].bar(res["Model"], res["RMSE"], color="#D62728")
    ax[1].set_title("RMSE in lakh (lower is better)")
    for a in ax:
        a.tick_params(axis="x", rotation=30)
        a.grid(True, axis="y")
    plt.tight_layout()


def plot_segments(df, seg, unusual):
    plt.figure(figsize=(8, 5))
    for s in range(3):
        m = seg == s
        plt.scatter(df.loc[m, "car_age"], df.loc[m, "price_lakh"], s=10, label=SEGMENT_NAMES[s])
    plt.scatter(df.loc[unusual, "car_age"], df.loc[unusual, "price_lakh"], color="black",
                marker="x", s=40, label="Unusual listing")
    plt.xlabel("Car age (years)")
    plt.ylabel("Price (lakh)")
    plt.title("Car Segments and Unusual Listings")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
```

Step 6: Run the project

Code

```
df = load_data()
print("Data shape:", df.shape)
state = train_models(df)
seg, unusual, km_model, sc2, remap = segments_and_outliers(df)
pack = (km_model, sc2, remap)
predict_car({"name": "Maruti Alpha", "year": 2018, "km": 45000, "fuel": "Petrol",
```

```
            "seller": "Individual", "transmission": "Manual", "owner": 1}, state, pack, df)
predict_car({"name": "BMW Beta", "year": 2020, "km": 30000, "fuel": "Diesel",
            "seller": "Dealer", "transmission": "Automatic", "owner": 1}, state, pack, df)
```

Output

```
cars.csv not found, using generated sample data
Data shape: (1462, 13)

Best K for k-NN (5-fold cross-validation): 3
Best AdaBoost settings: {'learning_rate': 0.5, 'n_estimators': 100}

Model comparison (price in lakh rupees):
                     Model    R2   MAE  RMSE
Multiple Linear Regression 0.832 0.546 1.261
     Polynomial Regression 0.969 0.309 0.544
             Decision Tree 0.918 0.516 0.883
          k-NN Regression 0.730 0.789 1.600
        AdaBoost Regressor 0.932 0.425 0.804

Best model by RMSE: Polynomial Regression

Car segments:
Budget: 454 cars, average price 0.70 lakh
Mid-range: 500 cars, average price 2.05 lakh
Premium: 508 cars, average price 5.21 lakh

Unusual listings found by DBSCAN: 22 of 1462

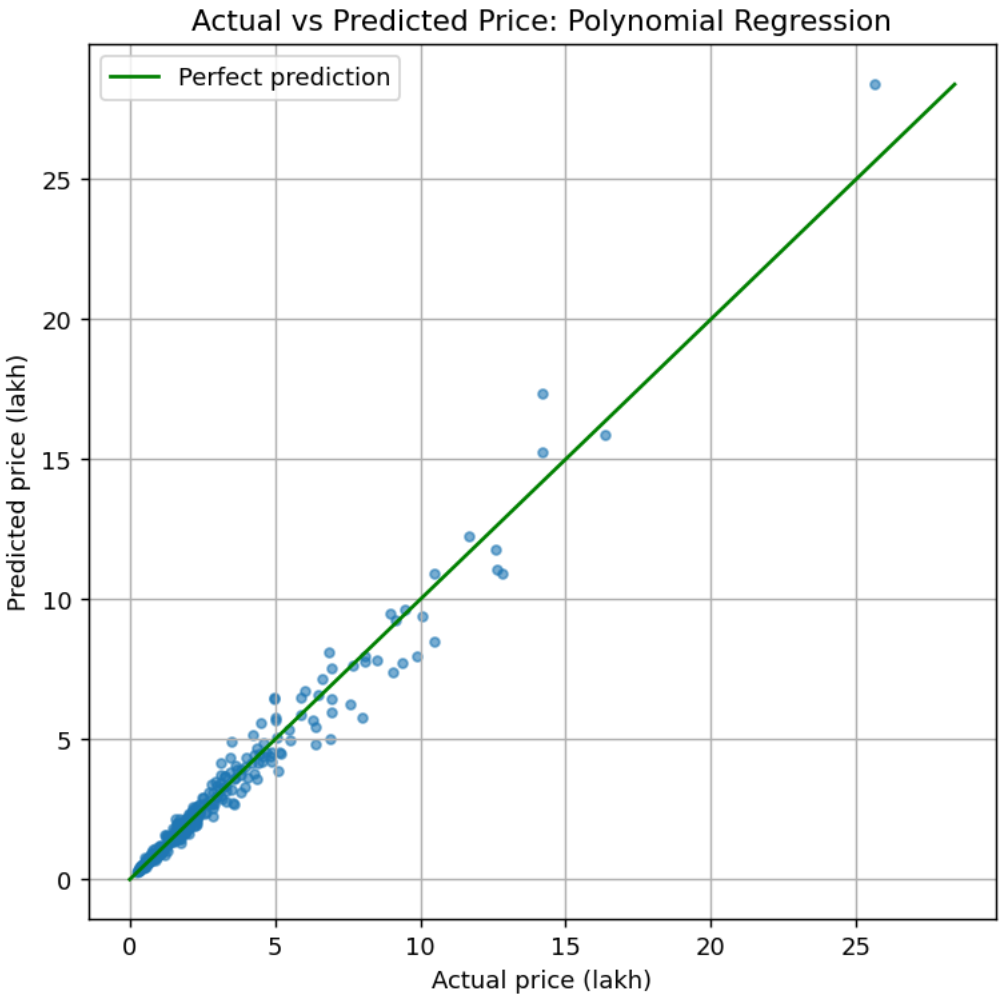
Car: Maruti Alpha 2018, 45000 km, Petrol, Manual
Predicted price: 1.65 lakh rupees (Polynomial Regression)
Segment: Mid-range
Three most similar cars in the data:
         name  year  km_driven  price_lakh
 Maruti Gamma  2017    63030.0        1.19
  Maruti Beta  2019    50882.0        1.69
Hyundai Gamma  2017    64802.0        1.76

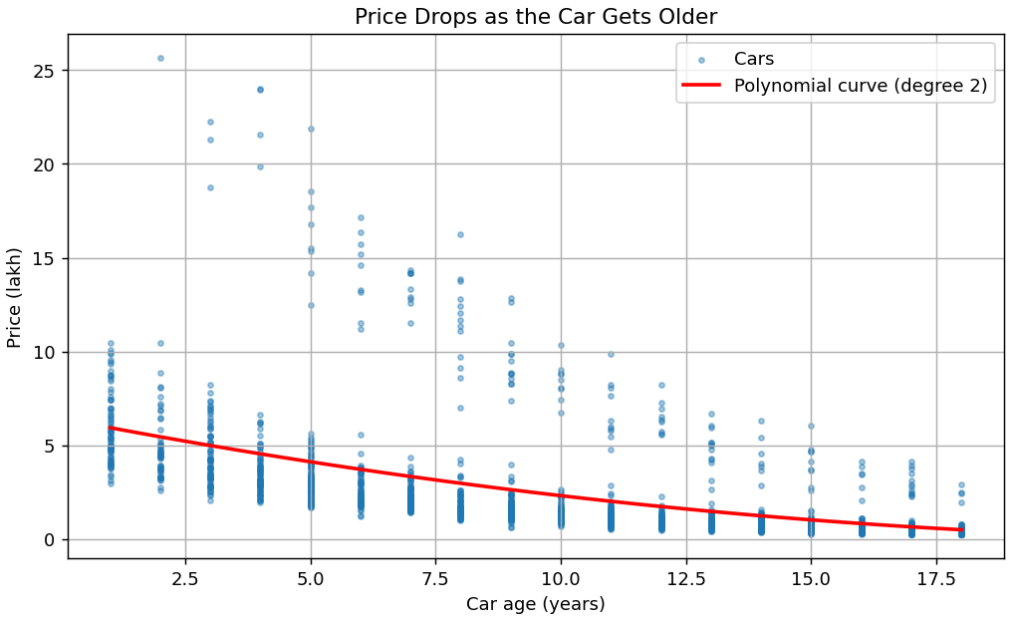
Car: BMW Beta 2020, 30000 km, Diesel, Automatic
Predicted price: 23.07 lakh rupees (Polynomial Regression)
Segment: Premium
Three most similar cars in the data:
          name  year  km_driven  price_lakh
     BMW Gamma  2011   109919.0        6.03
     Audi Beta  2022    40948.0       23.94
Mahindra Alpha  2019    49272.0        3.31
```

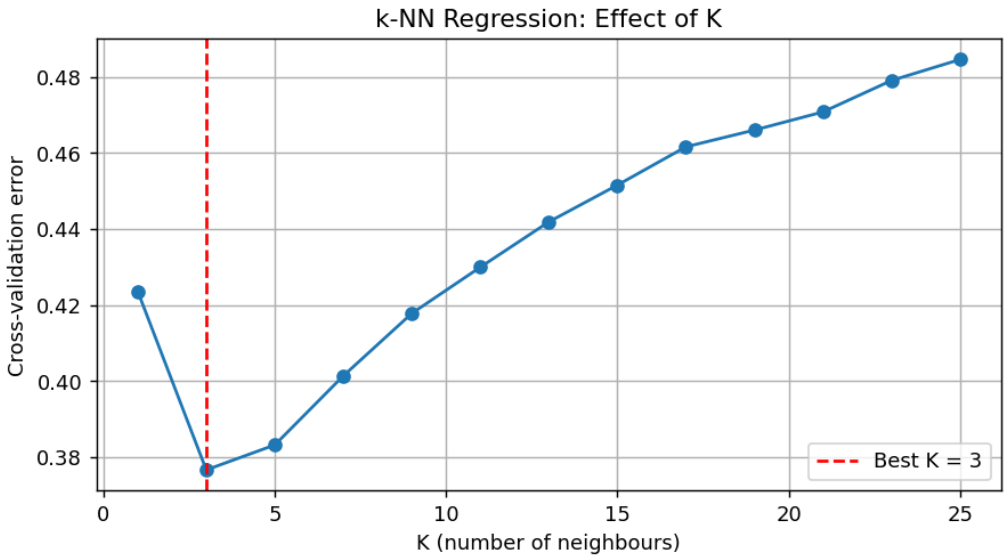
Step 7: Show the graphs

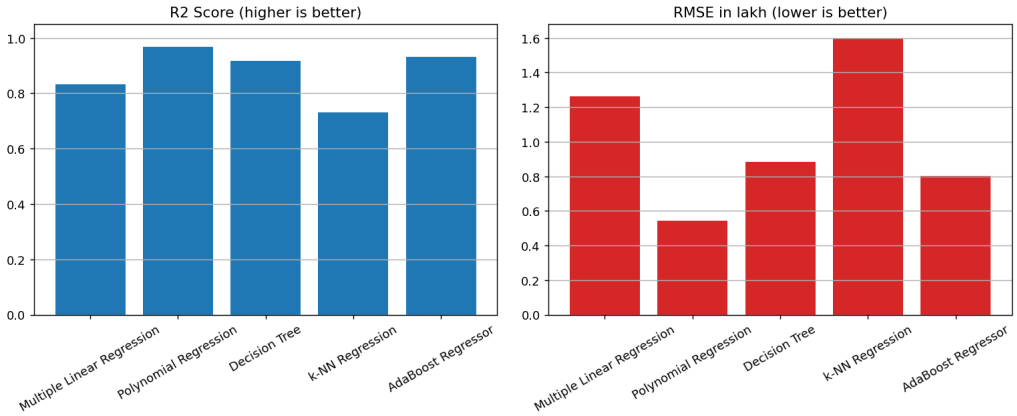
Code

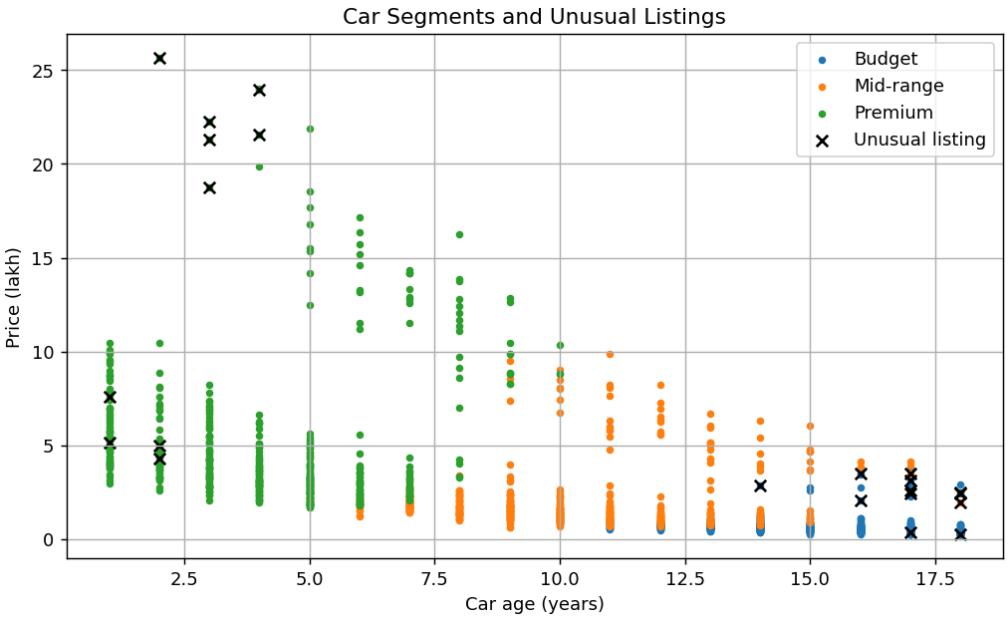
```
plot_actual_vs_pred(state)
plt.show()
plot_age_curve(df)
plt.show()
plot_k_curve(state)
plt.show()
plot_models(state["res"])
plt.show()
plot_segments(df, seg, unusual)
plt.show()
```


Actual vs Predicted Price: Polynomial Regression
Perfect prediction
Predicted price (lakh)
Actual price (lakh)
0
5
10
15
20
25


Price Drops as the Car Gets Older
Cars
Polynomial curve (degree 2)
Price (lakh)
Car age (years)
2.5
5.0
7.5
10.0
12.5
15.0
17.5


k-NN Regression: Effect of K
Cross-validation error
K (number of neighbours)
Best K = 3


R2 Score (higher is better)
RMSE in lakh (lower is better)
Multiple Linear Regression
Polynomial Regression
Decision Tree
k-NN Regression
AdaBoost Regressor


Car Segments and Unusual Listings
Budget
Mid-range
Premium
Unusual listing
Price (lakh)
Car age (years)